Data Report

The music_db is a PostgreSQL-based relational database developed to manage a collection of songs and their associated metadata. Development began in the middle of the second week of our project. The final schema, which includes the newly added sinus_vector column, was completed in the last week.

PostgreSQL was chosen as the database system because some prior knowledge of its usage already existed within the team.

Initially, the idea was to store all data in a single table, with one column per genre and a Boolean value indicating whether each song belonged to that genre. However, PostgreSQL has a limitation of 1600 columns per table, and since we had approximately 6200 genres, this approach was not feasible. Consequently, we revised our design to comply with PostgreSQL's constraints.

The final design of our database, which is similar to the first revision, consists of three tables: songs, genres, and song_genres. In the last week, the sinus_vector column was added to the songs table.

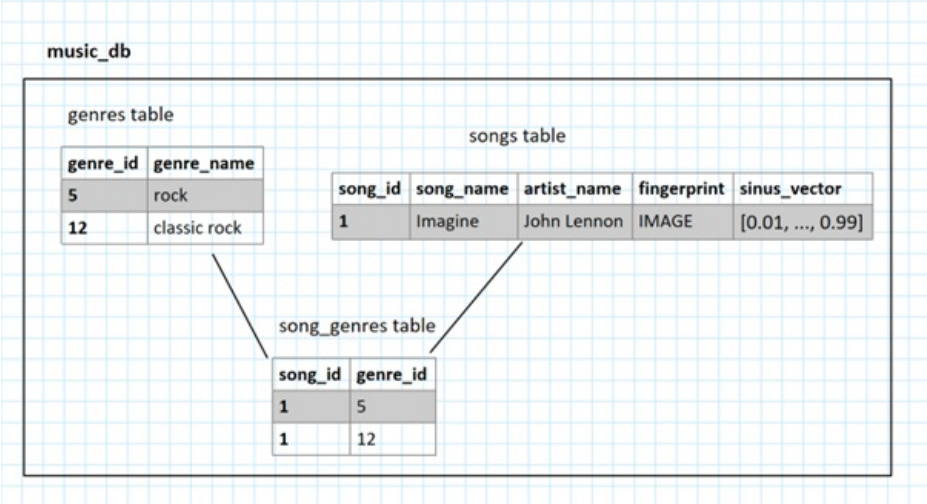

Table and Structure

1.Table:songs

Stores detailed information about every individual song.

Fields:

- song_id (int, primary key): Automatically generated unique identifier for each song
- song_name (varchar(1000), NOT NULL): The title of the song. Cannot be empty.
- o artist_name (varchar(1000), NOT NULL): The performing artist. Cannot be empty
- o fingerprint (bytea): A picture of a spectrogram of the song. This field is optional and can be added after song_name and artist_name.
- o sinus_vector (float array): A numerical array representing the song's sinus vectorisation of the spectrogram.

song_id	song_name	artist_name	fingerprint	sinus_vector
1	let it be	the beatles	IMAGE	[0.5, 0.1,...0.34]

2.Table: genres

Contains the list of all music genres assigned to songs, obtained from the scraped website.

Fields:

- genre_id (int, primary key): Automatically generated unique identifier for each genre.
- genre_name (varchar(1000)): The name of the genre (e.g., Rock, Classic Rock).

genre_id	genre_name
1	rock
2	pop
3	soft rock

3.Table: song_genres

A join table that enables many-to-many relationships between songs and genres.

Fields:

- song_id (foreign key): References songs.song_id.
- genre_id (foreign key): References genres.genre_id.

song_id	genre_id
1	1
1	2
1	3

Constraints of our Database

To make data insertion cleaner and data cleaning easier, we implemented several constraints:

Constraint

- unique_song_artist → It constraints the user to put in duplicates of a song. The song_name and artist_name have to be unique.
- at_least_a_genre → to insert a new song, it has to have at least one genre.
- song_name, artist_name = NOT NULL → every song needs a song and an artist name, otherwise it will be skipped and not inserted.

Connection to the Database

The database is hosted on the laptop of one of our team members. To enable access for others, a stable and secure connection was required.
We used Tailscale, a modern, secure, and easy-to-use VPN built on top of WireGuard, a fast and lightweight encryption protocol. Tailscale allows users to securely connect across the internet as if they were on the same local network, without complex configurations or port forwarding.

How Tailscale Works:

1. Each team member created a Tailscale account and installed the app on their computer.
2. The person hosting the database created a Tailscale network.
3. All team members were invited to join the network.
4. Everyone connected to the Tailscale network.
5. MagicDNS was enabled. This feature automatically registers domain names for devices in the tailnet, allowing users to connect via machine name instead of IP address — making the connection stable and consistent.
6. The pg_hba.conf file in PostgreSQL was updated to allow Tailscale connections.
7. Team members connected to the database using the Tailscale hostname.

With these steps, we were able to connect to the database using the machine name, which kept the connection stable. This eliminated the need to update IP addresses daily, saving us time and hassle.